System Design: Design Principles: Modularity, Scalability, Reliability and Availability, Load Balancing, Security, Performance Optimization, Interoperability and Maintainability

System Design(SD)(22CS024)

Bachelor of Engineering- Computer Science & Engineering

Department of Computer Science & Engineering

Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

Modularity

- Modularity in system design,, is the principle of breaking a system into smaller, independent modules, each handling a specific functionality.
- This approach enhances maintainability, testability, and scalability, as modules can be developed, tested, and updated independently without affecting the rest of the system.
- A key aspect is the use of well-defined interfaces for communication between modules, which simplifies complexity and facilitates organization.

Core principles and benefits

- **Decomposition:** Dividing a system into smaller, manageable components that perform a specific function.
- **Independence:** Modules should be able to operate independently with minimal impact on others, making it easier to manage and update them separately.
- **Standardized Interfaces:** Modules communicate through clearly defined interfaces, which documents expected behavior and simplifies integration.
- **Reusability:** Modules can be designed to be reused across different projects or parts of a system.
- **Interchangeability:** A module can be replaced with a new one as long as the interface remains the same, minimizing changes to the rest of the system.

Scalability

- Scalability refers to a system's ability to handle increasing load (e.g., users, data, transactions) without performance degradation.
- It ensures the system can adapt to growth and changing demands.
- Scalability can be achieved through horizontal scaling (adding more machines) or vertical scaling (upgrading existing machines).
- A scalable system can maintain acceptable performance levels even as the number of users and data volume increase.

Strategies for Achieving Scalability

- **Load Balancing**: Distributing incoming requests across multiple servers to prevent overloading a single resource.
- **Database Sharding**: Splitting a large database into smaller, manageable parts to improve performance and manageability.
- **Caching**: Storing frequently accessed data in high-speed memory to speed up response times.
- **Asynchronous Processing**: Offloading background tasks to separate workers, ensuring smooth and uninterrupted user experiences.

Horizontal and Vertical Scaling | System Design

In system design, scaling is crucial for managing increased loads. This article explores horizontal and vertical scaling, detailing their differences. Understanding these approaches helps organizations make informed decisions for optimizing performance and ensuring scalability as their needs evolve.

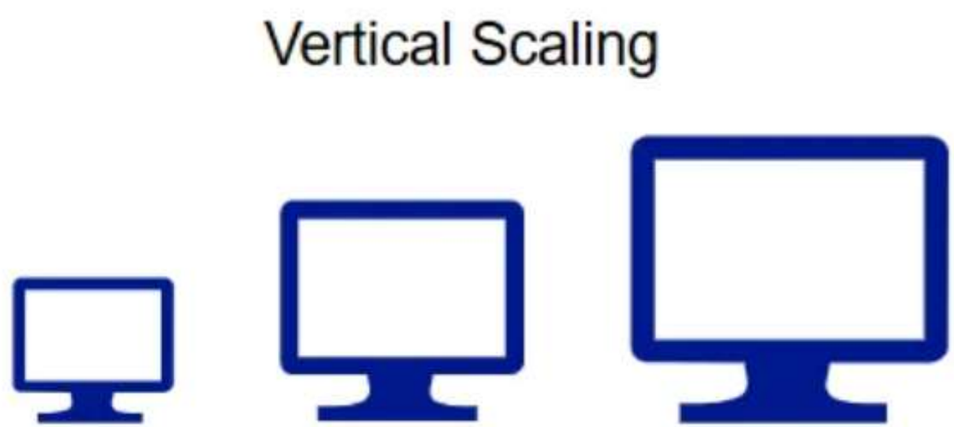
Horizontal scaling and **vertical scaling** are two different approaches to scaling a system, both of which can be used to improve the performance and capacity of the system.

What is Vertical Scaling?

Vertical scaling, also known as scaling up, refers to the process of increasing the capacity or capabilities of an individual hardware or software component within a system.

- You can add more power to your machine by adding better processors, increasing RAM, or other power-increasing adjustments.

- Vertical scaling aims to improve the performance and capacity of the system to handle higher loads or more complex tasks without changing the fundamental architecture or adding additional servers.



Characteristics of the Vertical Scaling

- This approach is also known as the ‘**scale-up**‘ approach.
- It doesn’t require any partitioning of data and all the traffic resides on a **single node with more resources**.
- Its implementation is easy.
- Less administrative effort as you need to manage just one system.
- Application compatibility is maintained.
- Mostly used in small and mid-sized companies.
- MySQL and Amazon RDS is a good examples of vertical scaling.

Advantages and Disadvantages of Vertical Scaling

CHITKARA UNIVERSITY

Advantages of vertical scaling

- **Increased capacity:** A server's performance and ability to manage incoming requests can both be enhanced by upgrading its hardware.
- **Easier management:** Upgrading a single node is usually the focus of vertical scaling, which might be simpler than maintaining several nodes.

Disadvantages of vertical scaling:

- **Limited scalability:** While horizontal scaling can be readily extended by adding more nodes and vertical scaling is constrained by the hardware's physical limitations.
- **Increased cost:** It may be more costly to upgrade a server's hardware than to add extra nodes.
- **Single point of failure:** One server still receives all incoming requests thus increasing the possibility of downtime in the event of a server failure.

Horizontal Scaling

- Horizontal scaling, also known as scaling out, refers to the process of increasing the capacity or performance of a system by adding more machines or servers to distribute the workload across a larger number of individual units.
- In this approach, there is no need to change the capacity of the server or replace the server.
- Also, like vertical scaling, there is no downtime while adding more servers to the network.

Horizontal Scaling

Characteristics of the horizontal scaling

- This approach is also known as the '**scale-out**' approach.
- Horizontal scalability can be achieved with the help of a distributed file system, clustering, and load–balancing.
- Traffic can be managed effectively.
- Easier to run fault tolerance.
- Easy to upgrade.
- Easy to size and resize according to your needs.
- Implementation cost is less expensive compared to scaling up.
- Google with its Gmail and YouTube, Yahoo, Facebook, eBay, Amazon, etc. are heavily utilizing horizontal scaling.
- Cassandra and MongoDB are good examples of horizontal scaling.

Advantages and Disadvantages of Horizontal Scaling

- **Advantages of horizontal scaling**
  - **Increased capacity:** More nodes or instances can handle a larger number of incoming requests.
  - **Improved performance:** By distributing the load over several nodes or instances, it is less likely that any one server will get overloaded.
  - **Increased fault tolerance:** Incoming requests can be sent to another node in the event of a node failure, lowering the possibility of downtime.
- **Disadvantages of horizontal scaling**
  - **Increased complexity:** Managing multiple nodes or instances can be more complex than managing a single node.
  - **Increased cost:** Adding more nodes or instances will typically increase the cost of the system.

Availability

- Availability refers to the percentage of time a system is accessible and operational, often expressed as uptime.

- It measures how frequently users can access and utilize the system.

- $\text{Availability (\%)} = \left(\frac{\text{Uptime}}{\text{Total Time}}\right) \times 100$

- High availability means minimal downtime and the ability to serve users consistently.

- Availability is crucial for systems where users expect constant access, like e-commerce websites or social media platforms.

- For instance, 99.9% uptime (also known as “three nines”) means that your system will experience downtime of just 8.77 hours per year. Maximizing uptime is key to delivering a seamless user experience.

Key Factors That Impact Availability

Single Points of Failure (SPOF): These occur when a single component failure can bring down the entire system. It's essential to design your system without SPOFs by including **redundancy** and **failover mechanisms**.

- **Fault Tolerance**: A fault-tolerant system continues to operate smoothly even if certain components fail. This ensures that the application remains functional despite minor issues.
- **Auto-Healing Infrastructure**: Automated tools that monitor your system and fix problems before they affect users.
- **Content Delivery Network (CDN)**: CDNs distribute content from multiple servers across the globe, reducing latency and speeding up load times for users regardless of location.

Reliability

- Reliability focuses on the consistency and correctness of a system's performance.
- A reliable system performs its intended functions as expected, even under various conditions and potential errors.
- It encompasses fault tolerance, meaning the system can handle unexpected issues without failing completely.
- Reliability is vital for systems where incorrect results or system failures have significant consequences, such as financial transactions or healthcare systems.

Building a Reliable System

- **Redundancy**: Redundant components or backups ensure that even if one part of your system fails, your application continues to function without data loss or major disruptions.
- **Database Replication**: Replicating data across multiple databases to safeguard against data corruption or loss.
- **Graceful Degradation**: When parts of the system fail, graceful degradation ensures that the application continues to function, instead of a complete crash.
- **Comprehensive Monitoring**: Tools like **Prometheus**, **Grafana**, and **New Relic** provide real-time insights into system performance, helping identify and fix issues early.
- **Automated Testing & Chaos Engineering**: Simulating failures through automated testing to assess how your system performs under stress and ensures reliability under various conditions.

Load Balancing

- Load balancing in system design, as described by GeeksforGeeks, is a crucial technique for distributing incoming network traffic or computational tasks across multiple servers or resources in a distributed system.
- Its primary goals are to optimize resource utilization, maximize throughput, minimize response time, and prevent any single resource from becoming overloaded.

- Key aspects of load balancing in system design:
- **Purpose:**
- **High Availability:** Ensures continuous service by directing traffic away from failing servers.
- **Scalability:** Allows systems to handle increasing traffic by distributing load across more resources.
- **Performance Optimization:** Improves response times and throughput by preventing bottlenecks at individual servers.

Types of Load Balancing

- **Static Load Balancing:** Uses pre-defined rules or algorithms (e.g., Round Robin, Weighted Round Robin) to distribute load, without considering real-time server conditions.
- **Dynamic Load Balancing:** Adapts to changing system conditions (server load, network traffic) by dynamically adjusting load distribution using algorithms like Least Connections or Least Response Time.

Load Balancing Algorithms

- **Round Robin:** Distributes requests sequentially to each server in the pool.
- **Weighted Round Robin:** Assigns requests based on server capacity or weight.
- **Least Connections:** Directs requests to the server with the fewest active connections.
- **Least Response Time:** Routes requests to the server with the quickest response time.
- **IP Hash:** Uses a hash of the client's IP address to ensure requests from the same client always go to the same server.

USE Cases & Challenges

- **Use Cases:**
- **Web Applications:** Distributing web traffic across multiple web servers.
- **Database Systems:** Balancing read/write operations across database replicas.
- **Microservices Architectures:** Routing requests to appropriate service instances.
- **Content Delivery Networks (CDNs):** Directing users to the nearest or fastest edge server.
- **Challenges:**
- **SSL Termination:** Managing SSL certificates and potential performance overheads with SSL decryption/encryption.
- **Session Persistence:** Ensuring a user's requests are consistently routed to the same server if session state is maintained on the server.
- **Complexity:** Implementing and managing load balancers in complex distributed systems.

Security

- With various threats like cyberattacks, Data Breaches, and other Vulnerabilities, it has become very important for system administrators to incorporate robust security measures into their systems. Some of the key reasons are given below:
- **Protection Against Cyber Threats:** Data Breaches, Hacking, DoS attacks, etc.
- **Data Privacy and Compliance:** Different measures like GDPR, HIPAA, and CCPA recommend organizations to protect user data.
- **Intellectual Property Protection:** Most companies often have their valuable intellectual property stored in their systems in a digital manner.
- **Preventing Financial Loss:** Not only just loss of data but some cyber attacks can be used to steal valuable information and passwords which then can be used to steal money.
- **Reputation Management:** Leaking of confidential data and user data can damage the reputation of the company severely.

- **Steps and ways to ensure the security of a system**
- **Authentication and Authorization:**
- **Authentication** is a crucial step to identify the person who is using that certain device. Some of the most popular ways of authenticating are
- Multi-Factor Authentication (MFA)
- Biometrics
- Emphasis on strong password policy
- Single Sign-On.
- **Authorization** is a process in which the authority of the particular user trying to access a system is checked. Some of the most renowed ways of authorization are
- Attribute Based Access Control
- Least Privilege Principle
- Review the Accesses
- Resource Based Authorization.

Data Encryption

- Data Encryption is also a crucial step to ensure the safety of any system. Some of the key aspects to ensure data encryption are
- Select strong encryption algorithm like AES
- Use secure protocols like HTTPS, TLS, SSL while transferring data from source to destination.
- Encrypt stored data using different encryption types like full-disk encryption.
- Create a secure key management system to generate, store and update encryption keys whenever necessary.
- Use End to End encryption for the user generated contents and other sensitive communications.

Secure Coding Practices

- Secure Coding Practices are necessary to safeguard a system from various types of Cyber Threats. Some of the Secure Coding Practices is given below:
- **Input validation:** It is necessary to always validate user inputs to prevent threats like SQL Injection and XSS (Cross Site Scripting).
- **Access Control:** Implement proper access control mechanism so restrict the user to only let them access data or information they are authorized to do.
- **Secure Authentication:** Use of secure authentication methods like MFA , Face Recognition etc.
- **Error Handling:** Use meaningful error messages and logging information to avoid potential leak of sensitive and confidential data publicy which could be used by hackers.
- **Regular Updates and Patch Management:** Updating the security components alongside libraries and patches is a must to safeguard the system from cyber attacks.

Privacy and Security

In today's interconnected world, **privacy and security** are critical concerns in system design.
Designers must integrate robust security measures at every stage of development to protect sensitive data and prevent unauthorized access.
Key strategies include:

- **Encryption**: Securing data in transit and at rest using cryptographic techniques (e.g., AES, RSA, TLS).

- **Authentication & Authorization**: Implementing mechanisms like OAuth, JWT, or multi-factor authentication (MFA) to verify user identities and enforce role-based access control (RBAC).

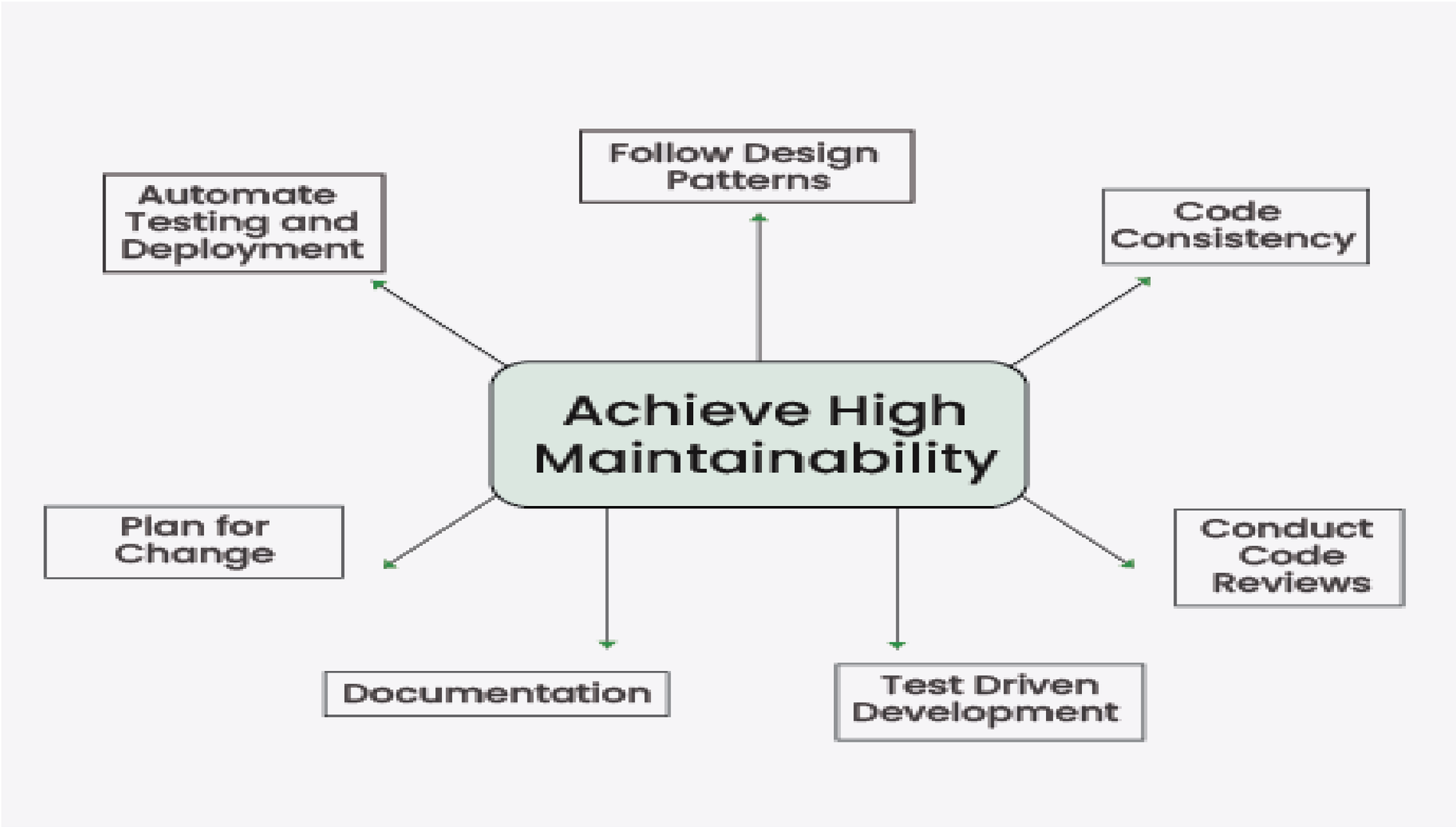
Maintainability

- Maintainability in system design refers to how easily a software system can be modified, updated, and extended over its lifespan.
- It's a crucial characteristic that ensures a system can adapt to changing requirements, fix bugs, and improve performance without significant disruption or cost.
- Good maintainability reduces long-term costs, minimizes the risk of introducing errors, and makes it easier for developers to understand, troubleshoot, and enhance the system.

Key Aspects of Maintainability:

- **Modularity**
- **Readability**
- **Testability**
- **Documentation**

How to achieve high Maintainability



Key Aspects of Maintainability:

Modularity:

- Breaking down the system into independent modules with well-defined interfaces makes it easier to modify or replace individual components without affecting the rest of the system.
- Design the system with a modular structure, where components are logically separated and have well-defined interfaces.
- This allows for independent development and modification of modules, making it easier to understand, test, and maintain specific parts of the system without impacting others.

Readability:

- Writing clear, concise, and well-documented code using consistent conventions makes it easier for developers to understand and work with the code.

Testability:

- Designing the system to support thorough testing (unit, integration, and automated) allows for quick identification and resolution of issues during modifications.

Documentation:

- Comprehensive and up-to-date documentation (design documents, API references, etc.) is essential for developers to understand the system's functionality and how to interact with it.

Why Maintainability is Important

Reduced Costs:

- Easier maintenance translates to lower long-term development and operational costs.

Faster Development Cycles:

- Modifying a maintainable system is quicker and more efficient, leading to faster feature delivery and bug fixes.

Improved Reliability:

- Well-maintained systems are more likely to be reliable because changes are less likely to introduce new errors.

Adaptability:

- A maintainable system can adapt to evolving business needs and technological advancements, ensuring its longevity.

- **Data Minimization & Anonymization**: Collecting only necessary data and anonymizing personally identifiable information (PII) to reduce privacy risks.

- **Secure APIs & Input Validation**: Preventing injection attacks (e.g., SQLi, XSS) through sanitization and parameterized queries.

- **Audit Logging & Monitoring**: Tracking access patterns and anomalies to detect and respond to breaches proactively.

Interoperability

- Interoperability in system design, particularly in distributed systems, refers to the ability of diverse systems or components to communicate, exchange data, and work together seamlessly, irrespective of their underlying technologies, programming languages, operating systems, or hardware.

Key aspects of interoperability in system design

- **Communication Protocols:**
- Establishing common rules and conventions for data exchange between systems. This ensures that systems can understand and process information from each other. Examples include HTTP/S, TCP/IP, and various messaging protocols.
- **Data Formats:**
- Defining standardized ways to represent and structure data exchanged between systems. This ensures that data is interpreted consistently by all participating systems. Examples include JSON, XML, and various industry-specific data standards.
- **Interfaces:**
- Providing well-defined points of interaction that allow systems to access each other's functionalities. APIs (Application Programming Interfaces) are a common mechanism for achieving this.

- **Middleware:**
- Utilizing intermediate software layers (e.g., API gateways, message brokers, Enterprise Service Buses) to facilitate communication and data transformation between disparate systems, bridging technological differences.
- **Semantic Interoperability:**
- Ensuring that the meaning and context of exchanged data are understood consistently across all systems. This goes beyond mere technical compatibility and focuses on shared understanding of information.

Importance in System Design:

- **Improved System Reliability:**
- Ensures smooth interaction between components, minimizing errors and failures.
- **Enhanced User Experience:**
- Prevents disruptions and inconsistencies when users interact with systems or third-party services.
- **System Compatibility:**
- Enables software to function correctly across different platforms and environments.
- **Meeting Industry Standards:**
- Adherence to specific interoperability standards, particularly in regulated industries like healthcare or finance.
- **Facilitates Integration:**
- Allows for easier integration of new systems or components into an existing ecosystem.
- **Promotes Reusability:**
- Enables components or services to be reused across different applications and systems.

- The ability to design systems that are not only functional but also optimized for **performance and scalability** is essential. As systems grow in complexity, the need for effective optimization techniques becomes increasingly critical.
- **Data Structures & Algorithms**
- Choose data structures (hash tables, trees, arrays) wisely for efficiency, memory usage, and scalability.
- **Google Search** does a mapping from keywords to document lists, a form of hash table optimized for search queries.

Caching

- Use caching (in-memory, CDN, multilevel) to reduce latency and backend load.
- We can always distribute our items in categories, like static, moderate and dynamic and add high caching expiration time for static content like images, CSS and JS files, moderate caching for items like list of items and low on things like messages.

- Caching needs to be done at every level like Browser Caching, Database Caching, CDN Caching and in-memory caching.

Database Optimizations

- Use Indexing in databases for frequently accessed items.
- Use Query Optimizations to reduce response times.
- **Netflix** employs CDNs to cache video locally and cache personalized recommendations. **Twitter** uses Redis to cache timelines and sessions, enabling real-time updates and handling millions of queries per second.
- **Facebook** leverages Memcached and to cache frequently accessed social graph data (like friendships), reducing database stress.

Scalability & Load Balancing

- Use **horizontal scaling** (adding more nodes) for cost-effective growth; **vertical scaling** (upgrading resources) has limits
- Implement **load balancing** strategies—such as round-robin, least-connections, or IP-hash—to distribute traffic evenly.
- For example, Netflix dynamically adjusts service clusters based on real-time CPU, network, and queue. They do anticipatory scaling—spinning up thousands of VMs within minutes during traffic spikes—and decommissioning them as demand falls.

Microservices & Architectural Patterns--Network Optimization:

- **Microservices & Architectural Patterns-**
- Break systems into independent components to improve scaling, fault isolation, and developer agility.
- **Netflix** runs over 700 microservices, enabling thousands of daily deployments with robust scalability **Uber** transitioned from monolithic to microservices to handle global scale dynamically.
- **Network Optimization:**
- Reduce the number of HTTP requests by bundling assets, using CSS sprites, and leveraging browser caching.
- Compress images and use appropriate file formats to reduce their size and improve page load times.
- Use CDNs to distribute static assets geographically, reducing latency and improving content delivery speed

Thankyou